

19 — Simulare un gioco (e sapere dove fermarsi)

La domanda a cui risponde: quanto di un gioco di carte si può simulare, e come si decide il confine? Più in generale: cosa fai quando metà dei tuoi dati è **strutturata** (numeri, tipi, costi) e metà è **prosa** scritta per un essere umano?

Per contorno: una macchina a stati che si prova senza browser, animazioni che non possono mentire, e perché il caso ha un seme.

Il fatto

L'app sapeva dire *quali carte mettere nel mazzo, non come si gioca*. La v3 della roadmap è la mini partita: due mazzi salvati che si affrontano davvero, con le regole della casa attive per quel mazzo.

Il primo istinto — “simuliamo il TCG” — è sbagliato in due modi. È troppo (le regole complete sono un libretto, e le abilità sono testo libero) ed è troppo poco (senza debolezza e stati speciali non insegna la strategia di base). Il lavoro vero è stato **decidere il confine e dichiararlo**.

Parte 1 — Dati strutturati e dati in prosa

Guardando una carta con occhi da programma, i campi si dividono nettamente:

Strutturato	In prosa
PS, tipo, stadio, ritirata costo e danno degli attacchi debolezza ($F_{\text{UOCO}} \times 2$), resistenza	testo dell'attacco effetto degli Allenatori abilità e poteri

Il primo gruppo si simula esattamente: sono numeri e chiavi. Il secondo no — “*Il tuo avversario mostra le carte che ha in mano e tu peschi una carta per ogni carta Allenatore presente tra quelle carte*” non è eseguibile da niente che si possa scrivere in un pomeriggio.

E in mezzo c'è una terza categoria che è la più interessante: **prosa che contiene un vocabolario chiuso**.

Gli stati speciali sono prosa, ma non del tutto

Veleno, bruciatura, sonno, paralisi, confusione sono cinque, si scrivono sempre con le stesse parole (“*viene addormentato*”, “*è ora Avvelenato*”) e le loro regole non stanno sulla carta: stanno sul regolamento, uguali per tutte.

Quindi il riconoscimento è una ricerca di cinque radici — *avvelenat*, *bruciat*, *addormentat*, *paralizzat*, *confus* — e tutto il resto è codice normale:

```
export function riconosciStati(attacco) {  
  const testo = String(attacco?.effetto ?? '').toLowerCase();  
  return RADICI.filter(([, radice]) => testo.includes(radice)).map(([stato]) => stato);  
}
```

Radici e non parole intere perché la carta declina (*avvelenato*, *avvelenata*, *avvelenati*): tagliare prima della desinenza copre tutti i casi senza elencarli.

La parte fragile è confinata a cinque stringhe. Se domani TCGdex cambia il modo di scrivere, si rompe lì e si aggiusta lì — non in mezzo alla logica dei turni.

Parte 2 — Il confine dichiarato

Per gli Allenatori la scelta è la stessa idea portata più in là: si riconoscono poche **formule intere**, e tutto il resto si dichiara.

```
const pesca = piatto.match(/^pesca ([a-z]+\d+) cart[ae]\.?$/);
```

Nota l'ancoraggio `^...$`: la formula si riconosce solo se è **tutta** la frase. “Pesca tre carte.” sì; “...e tu peschi una carta per ogni carta Allenatore...” no, e infatti quella carta esce come `{tipo: 'manuale'}`.

È una scelta contro-istintiva: un riconoscitore più generoso “funzionerebbe” su più carte. Ma **una simulazione che sbaglia in silenzio insegna la regola sbagliata**, ed è il danno peggiore possibile in un'app che serve a imparare. Un bambino che vede l'app pescare due carte quando la carta ne dava tre non impara “l'app ha un bug”: impara la regola sbagliata.

Quindi le carte non riconosciute **si giocano lo stesso**, mostrando il testo:

```
mosse.push({ tipo: 'allenatore', ..., aMano: effetto.tipo === 'manuale', testo: effetto.testo });
```

E l'avviso arriva **prima** di giocarla, non dopo: scoprire a cose fatte che dovevi applicarla tu è una sorpresa, e le sorprese in un gioco che insegna sono tutte cattive.

Leggere la carta e applicarla, del resto, è esattamente quello che dovranno fare col mazzo fisico in mano. Non è un ripiego: è il gioco.

Parte 3 — Una macchina a stati che si prova senza browser

`engine/partita.js` non tocca il DOM, non tocca IndexedDB e **non chiama** `Math.random()`: il caso arriva da `engine/casuale.js` con un seme.

Il motivo è pratico. Una partita è tutta casi limite:

- la mano iniziale senza nessun Pokémon Base;
- il KO che prende l'ultimo Premio e chiude la partita;
- l'addormentato che non può ritirarsi;
- il mazzo che finisce mentre peschi.

Giocando a mano se ne incontra uno ogni venti partite. In un test si scrivono in sei righe:

```
test('pescare a mazzo vuoto fa perdere la partita', () => {  
  let s = conAttivi(partitaDiProva());  
  s.giocatori[0].mazzo = [];  
  s = pesca(s);  
  assert.equal(s.vincitore, 1);  
});
```

E il seme rende la partita **ripetibile**: due `iniziaPartita` con lo stesso seme danno la stessa mano. Serve ai test, e servirà a chi vorrà rivedere una partita già giocata.

I test hanno già ripagato due volte mentre scrivevo:

1. la **mano senza Base** — l'avevo semplicemente dimenticata, e il test l'ha trovata al primo giro (nel gioco vero si rimescola: è il *mulligan*);
2. la preparazione in cui l'avversario **non schierava mai**, perché in fase di preparazione nessuno passava la mano.

Il secondo è istruttivo: era un difetto che i test non potevano trovare, perché nei test ero io a passare la mano a mano. L'ha trovato l'app al primo avvio. Test e prova sul campo trovano bug diversi, e nessuno dei due sostituisce l'altro.

Un NaN che valeva zero

Il danno di un attacco sulle carte non è quasi mai un numero pulito: “20+”, “30×”, “10-”. `Number("20+")` è NaN, e NaN finiva in un `|| 0`: **zero danni**, in silenzio. L'attacco si giocava, l'animazione partiva, e non succedeva niente.

```
export function dannoStampato(valore) {  
  const cifre = String(valore ?? '').match(/\d+/);
```

```
return cifre ? Number(cifre[0]) : 0;
}
```

Si tiene la parte fissa; il “+” dipende quasi sempre da un lancio di moneta o da quante Energie hai addosso — cose che stanno scritte nell’effetto, che la partita mostra. È lo stesso confine della Parte 2, applicato a un numero.

Parte 4 — Animazioni che non possono mentire

<tavolo-partita> non anima *quando gli si dice di animare*: guarda il **registro** della partita, confronta le righe nuove con quelle già mostrate e anima la differenza.

```
const nuove = s.registro.slice(this.#registroMostrato);
this.#registroMostrato = s.registro.length;
for (const evento of nuove) {
  if (evento.tipo === 'attacco') this.#animaAttacco(evento);
}
```

La conseguenza è che **un’animazione non può raccontare una cosa diversa da quella che il motore ha fatto**: se il registro non dice “attacco”, nessuno trema. È la stessa idea del data-attribute che porta l’informazione invece di farla dedurre: la verità sta in un posto solo, e la vista la legge.

Le animazioni sono CSS puro — l’attaccante scatta, il difensore trema, il numero del danno sbuca e sale sfumando, l’esausto sbiadisce in grigio — e stanno tutte dietro prefers-reduced-motion, che le spegne lasciando i numeri.

Un dettaglio che vale più di tutte: **le mosse impossibili restano a schermo**, spente, con scritto perché.

Quick Attack (20+ danni)
Servono 2 Energie, ne hai 0.

Un comando che sparisce non insegna niente; uno spento che non spiega insegna solo a diffidare. Questa è la differenza fra un gioco e un gioco che insegna.

Domande di verifica

1. **Il vocabolario chiuso.** Gli stati speciali si riconoscono perché sono cinque e scritti sempre uguale. Trova nel dataset un’altra informazione in prosa che ha la stessa proprietà, e di’ come la riconosceresti.
2. **Il riconoscitore generoso.** Togli l’ancoraggio `^...$` dalla formula di pesca e cerca nel dataset tre carte che verrebbero eseguite male. Cosa imparerebbe di sbagliato chi gioca?
3. **Il seme.** `iniziaPartita` accetta `seme`. Scrivi un test che dimostri che due partite con semi diversi divergono, e uno che dimostri che con lo stesso seme restano identiche fino alla fine.
4. **Il registro come verità.** Aggiungi un evento nuovo al motore (per esempio `mossa-annullata`) e fai in modo che il tavolo lo racconti. Quante righe hai dovuto toccare, e in quanti file?
5. **Il confine.** Elenca tre cose del TCG che questa partita **non** simula. Per ciascuna, di’ se le manca un *dato* o un *pezzo di motore* — e quale delle due mancanze è più facile da colmare.